Assignment 03

Arrays & Algorithm Analysis

TKU211231 — Data Structures and Algorithms

Syukron Abu Ishaq Alfarozi
Department of Electrical and Information Engineering
Universitas Gadjah Mada

Due Date: One week from assignment date
Total Points: 100 points

Instructions

Read carefully before starting:

1. All code must compile with `g++ -std=c++17 -Wall -Wextra`
2. Include a compilation command in a comment at the top of each source file
3. All **written analysis** (Problems 1 and 4) must be submitted as a PDF or neatly handwritten scan; no plain text files
4. For coding problems, test your code thoroughly before submission
5. Submit **all** source files (`.cpp`) and a `README.txt`
6. Follow the naming convention: `problem#.cpp` (e.g., `problem2.cpp`)
7. Memory leaks will result in point deductions; test with `g++ -g -fsanitize=address`
8. Late submissions: 10% penalty per day, maximum 3 days

Problem 1: Algorithm Analysis (25 points) — Written

For each code snippet below, perform a **complete step-by-step analysis**:

(i) **Exact operation count** $T(n)$: annotate every line with its cost per execution and its total number of executions, then sum to obtain a polynomial in n.

(ii) **Big-O class**: apply asymptotic simplification rules to your $T(n)$ (drop lower-order terms, drop constant factors).

(iii) **Best/worst case**: state whether the best and worst cases differ, and explain

why.

Use the standard cost model from lecture: every assignment, comparison, arithmetic operation, and array access each cost 1 unit.

Snippet A (8 points)

```
// n >= 1
int sum = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < i; j++) {
        sum += arr[i] + arr[j];
    }
}
```

Hint: The inner loop runs $0, 1, 2, \ldots, n-1$ times for each value of i. Use $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$.

Snippet B (8 points)

```
// n >= 1; arr is a sorted array of n integers
int lo = 0, hi = n - 1, target = 42;
bool found = false;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == target)      { found = true; break; }
    else if (arr[mid] < target)  { lo = mid + 1; }
    else                         { hi = mid - 1; }
}
```

Hint: Each iteration halves the search space. How many times can you halve n before reaching 1? Express $T(n)$ in terms of $\lfloor \log_2 n \rfloor + 1$.

Snippet C (9 points)

```
// n >= 1
int count = 0;
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        for (int k = 0; k < n; k += 2) {   // note: k += 2
            if (arr[i] != arr[j] && arr[k] > 0)
                count++;
        }
    }
}
```

Hint: The innermost loop runs $\lceil n/2 \rceil$ times regardless of i and j. The middle loop runs a total of $\frac{n(n-1)}{2}$ times (summed over all i).

Problem 2: DynArray — Procedural Implementation (35 points)

Implement a resizable array using a plain `struct` and free functions (no classes, no templates, no `std::vector`).

Given structure (copy this exactly):

```
struct DynArray {
    int* data;       // pointer to heap-allocated buffer
    int  size;       // number of elements in use
    int  capacity;   // total allocated slots
};
```

Implement the following 8 functions:

1. `void initArray(DynArray& a)`
   Initialise with `capacity = 2`, `size = 0`, and allocate the heap buffer.
2. `void freeArray(DynArray& a)`
   Release the heap buffer and reset `size` and `capacity` to 0.
3. `void pushBack(DynArray& a, int value)`
   Append `value`. If `size == capacity`, double the capacity first (allocate new buffer, copy, free old, update pointer and capacity).
4. `void insertAt(DynArray& a, int index, int value)`
   Insert `value` at position `index` (0-based), shifting elements at positions $\geq$ `index` one slot to the right. Grow if necessary. Assume $0 \leq$ `index` $\leq$ `size`.
5. `void removeAt(DynArray& a, int index)`
   Remove the element at position `index`, shifting subsequent elements one slot to the left. Decrement `size`. Assume $0 \leq$ `index` $<$ `size`.
6. `int getAt(const DynArray& a, int index)`
   Return the element at `index`. Assume the index is valid.
7. `void setAt(DynArray& a, int index, int value)`
   Set the element at `index` to `value`. Assume the index is valid.
8. `void printDynArr(const DynArray& a)`
   Print in the format: `[10, 20, 30] (size=3, cap=4)`

In `main()`, demonstrate all functions in this order:

```
DynArray arr;
initArray(arr);

// 1. pushBack until a resize is triggered
for (int i = 1; i <= 5; i++) pushBack(arr, i * 10);
printDynArr(arr);    // [10, 20, 30, 40, 50] (size=5, cap=8)

// 2. insertAt
insertAt(arr, 2, 25);
```

```
printDynArr(arr);    // [10, 20, 25, 30, 40, 50] (size=6, cap
    =8)

// 3. removeAt
removeAt(arr, 0);
printDynArr(arr);    // [20, 25, 30, 40, 50] (size=5, cap=8)

// 4. getAt / setAt
cout << "Element at index 2: " << getAt(arr, 2) << "\n";   //
    30
setAt(arr, 2, 99);
printDynArr(arr);    // [20, 25, 99, 40, 50] (size=5, cap=8)

freeArray(arr);
```

Grading:

- `initArray` / `freeArray` (4 pts)
- `pushBack` with resize (8 pts)
- `insertAt` (7 pts)
- `removeAt` (7 pts)
- `getAt`, `setAt`, `printDynArr` (5 pts)
- No memory leaks (4 pts)

State the time complexity of each function in a comment directly above its implementation (`// O(?)`).

Problem 3: Linear Search vs. Binary Search (20 points)

Part A — Implement (10 points):
In a single file `problem3.cpp`, implement:

1. `int linearSearch(int* arr, int n, int target)`
   Returns the index of the first occurrence of `target`, or -1 if not found. Must use a loop (no `std::find`).

2. `int binarySearch(int* arr, int n, int target)`
   Returns the index of `target` in a **sorted** array, or -1 if not found. Iterative implementation only (no recursion). Use the midpoint formula `mid = lo + (hi - lo) / 2`.

3. `int* generateSortedArray(int n)`
   Allocates and returns a sorted array of n distinct integers (e.g., $\{0, 2, 4, \ldots, 2(n-1)\}$). The caller is responsible for freeing the returned pointer.

In `main()`, demonstrate on arrays of sizes $n \in \{10, 100, 1000\}$:

- Search for a value that **exists** (the element at the midpoint)
- Search for a value that does **not** exist
- Print the index returned by each algorithm and confirm they agree

Part B — Complexity Comparison Table (10 points):
Complete the following table **by hand** (submit as part of your written PDF):

Operation	Best Case	Worst Case	Average Case
Linear search (unsorted)			
Linear search (sorted, early exit)			
Binary search (sorted)			

For each cell, write the Big-O class *and* give a one-sentence explanation. Then answer: **At what minimum array size n does binary search become more efficient than linear search in the worst case?** Show your reasoning.

Problem 4: Amortized Analysis of pushBack (20 points) — Written

Consider the `pushBack` function from Problem 2, which doubles the capacity whenever `size == capacity`, starting from `capacity = 2`.
Part A — Trace (8 points):
Complete the table below by tracing 12 consecutive `pushBack` calls on an initially empty `DynArray`. For each call, record: the value pushed, the resulting `size`, the resulting `capacity`, and the **cost** of that specific call (cost = 1 for normal append; cost = 1 + old size when a resize occurs).

Call #	Value pushed	size after	capacity after	Cost
1	10			
2	20			
3	30			
4	40			
5	50			
6	60			
7	70			
8	80			
9	90			
10	100			
11	110			
12	120			

Part B — Total cost (4 points):

1. Sum all values in the Cost column. Call this T_{12}.
2. What is the *average* cost per call, i.e., $T_{12}/12$?
3. If we label the n-th call's cost c_n, write the full formula for the **total copy work** done across all resizes during n pushbacks. Identify this as a geometric series and evaluate it. What does this tell you about the amortised cost per call?

Part C — Growth factor comparison (8 points):
Suppose instead of doubling the capacity, `pushBack` grows by adding a **fixed amount** $k = 4$ (i.e., new capacity = old capacity +4).

1. Redo the trace from Part A (12 calls, starting from capacity 2) under this new strategy. Record the same four columns.
2. Compare the total cost of both strategies.
3. For a general sequence of n pushbacks, derive $T(n)$ for both the *doubling* and the *fixed-increment* strategies. Show that doubling gives $O(n)$ total whereas fixed-increment gives $O(n^2)$ total. Express the amortised cost per call in each case.

Grading Rubric

Component	Points	Criteria
Problem 1	25	Exact $T(n)$, Big-O, best/worst case reasoning
Problem 2	35	Correct functions, resize logic, no leaks
Problem 3	20	Search implementations, complexity table
Problem 4	20	Amortized trace, total cost, growth strategy proof
Total	**100**	

Additional point deductions:

- Compilation errors: -5 points per coding problem
- Memory leaks: -5 points per coding problem
- Missing complexity annotations (`// O(?)` comments): -2 points per function
- Incorrect output format: -2 points per coding problem
- Late submission: -10 points per day (max 3 days)

Submission Guidelines

What to submit:

1. **Source files:** `problem2.cpp`, `problem3.cpp`
2. **Written analysis PDF:** answers to Problems 1, 3B, and 4 (scanned handwriting is acceptable if legible)
3. **README.md** containing:
   - Your name and student ID
   - Brief description of each solution
   - Compilation commands for each source file
   - Any known issues or limitations
   - Time spent on the assignment

Submission method — GitHub + eLok UGM:

1. **Create a GitHub repository** named `StudentID-Assignment03` (e.g., `12345678-Assignment03`). The repository must be **public**.
2. **Push all required files** to the repository:
   - `problem2.cpp`
   - `problem3.cpp`
   - Written analysis PDF (Problems 1, 3B, and 4)
   - `README.md`
3. **Verify** that all files are accessible on GitHub before submitting the link.
4. **Submit the repository URL** on **eLok UGM** (https://elok.ugm.ac.id) under the Assignment 03 submission page. The URL should follow the format:

   `https://github.com/`*`yourusername`*`/StudentID-Assignment03`
5. Submissions via any other method (ZIP e-mail, direct file upload, etc.) will **not** be accepted.

Academic integrity reminder:

- This is an **individual assignment**
- You may discuss concepts but **not share code or written derivations**
- Submitting another person's analysis or code is plagiarism
- Violations result in zero points and disciplinary action

Tips for Success

1. **Problem 1**: Work through operation counting slowly—annotate every single line. Draw a table with columns: line, cost/execution, number of executions, total cost. Then sum column 4.

2. **Problem 2**: Implement and test each function in isolation before combining. Always test `pushBack` to the exact capacity boundary to make sure the resize is triggered at the right moment.

3. **Problem 3**: Print the index returned by *both* algorithms on the same input to cross-validate. Binary search only works on a **sorted** array—passing an unsorted array is undefined behaviour.

4. **Problem 4 Part C**: To derive the total cost for the fixed-increment strategy, note that resizes occur at sizes $2, 6, 10, 14, \ldots$ and each resize at capacity c copies c elements. This is an arithmetic series.

5. **Memory management**: Every `new[]` must be paired with a `delete[]`. Use `g++ -g -fsanitize=address` to catch leaks and out-of-bounds accesses before submission.

6. **Start early!** The written problems (1, 3B, 4) take more time than they appear—leave at least two dedicated sessions for them.

Good luck! Mastering Big-O analysis now will pay dividends
every time you design or choose an algorithm for the rest of the course.